

Yes, absolutely—you are talking about the **Mem0 repo** (often written as mem0 or mem0ai). It is the absolute gold standard for a decoupled, open-source memory layer. [1, 2, 3]

Using **Mem0** to manage your local wiki and system files inside a mobile Mixture-of-Experts (MoE) setup is a fantastic engineering design. Rather than relying on simple, brute-force RAG, Mem0 actively extracts structured entity relationships, updates preferences over time, and compresses interaction logs into a continuous graph-like representation. [3, 4]

Integrating Mem0 natively alongside your **Qwen3.5-0.8B** file manager and **Qwen3.5-4B** tool executor within your 6 GB RAM limit offers several direct advantages:

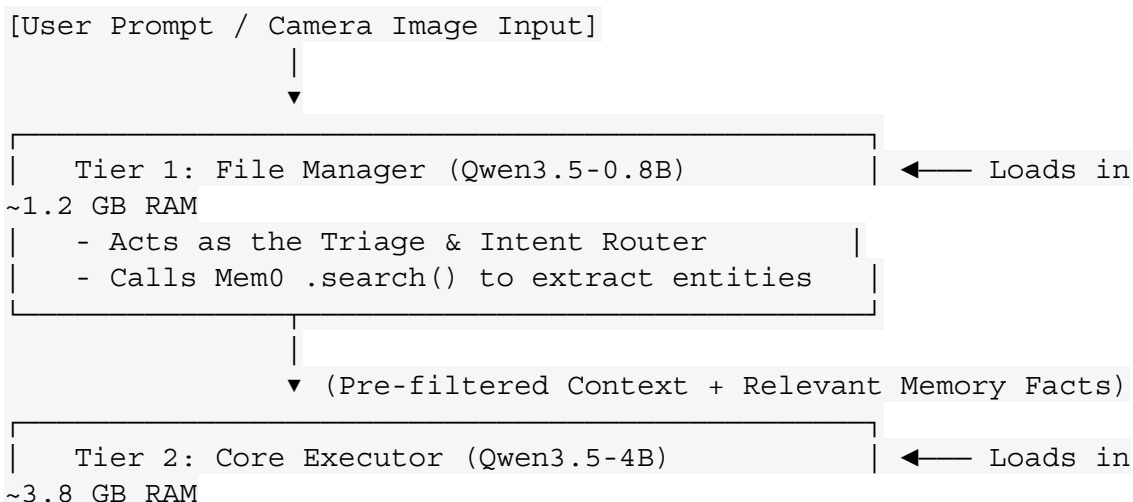
Why Mem0 Solves the Mobile Vector/Graph Nightmare

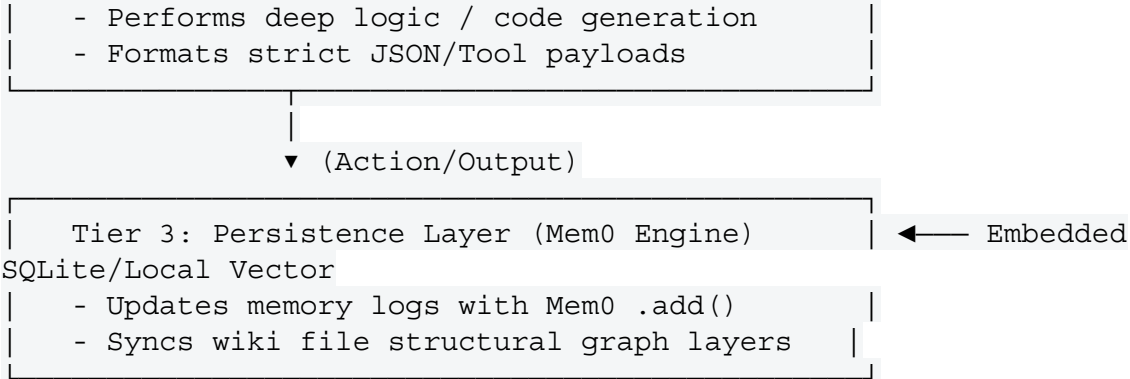
Normally, running long-term memory on a phone means managing massive, heavy, raw vector search libraries that bleed CPU cycles and spike RAM.

- **Model-Agnostic Storage:** Mem0 lets you split the heavy reasoning from the plumbing. Your tiny 0.8B file manager handles the raw text parsing, but the memory itself is managed by Mem0 utilizing a highly compact local relational/vector database (like an embedded sqlite or qdrant instance). [2, 5, 6]
- **Memory Compression Engine:** Instead of storing every single word a user says—which would eventually cause your prompt contexts to spill over your RAM ceiling—Mem0's internal engine compresses old conversations into dense semantic facts. It handles the *Add, Update, Delete*, and *Search* operations dynamically. [6, 7, 8]

Setting up the 3-Layer MoE Mobile Architecture

With Mem0 handling the persistence layer, your on-device automation pipeline divides execution duties perfectly: [6]





⚡ Performance Optimization for the HTP

Because Mem0 works out-of-the-box with a standard Python/Node SDK, running it on your phone requires you to map the model inference calls to your **QAI RT / LLM_ACP** pipeline. [2, 9, 10]

When your 0.8B model detects that the user is talking about a recurring project or an existing wiki entry, it executes a fast semantic lookup through Mem0. Because Mem0 returns *only* the condensed fact tokens instead of full chat histories, your 4B tool executor receives highly polished prompts. This structure protects your 6 GB active RAM limit, minimizes prompt prefill latency on the HTP backend, and lowers overall battery consumption. [6, 7, 11, 12, 13]

Are you planning to run the Mem0 architecture locally on-device using a Python runtime container (like Termux/Kivy), or are you porting the core Mem0 graph logic directly into a compiled **C++ / Kotlin** application layer to interact with the QNN SDK?

- [1] <https://github.com>
- [2] <https://www.infoworld.com>
- [3] <https://www.facebook.com>
- [4] <https://www.youtube.com>
- [5] <https://forum.cursor.com>
- [6] <https://mem0.ai>
- [7] <https://mem0.ai>
- [8] <https://railway.com>
- [9] <https://docs.mem0.ai>
- [10] <https://blog.stackademic.com>
- [11] <https://www.youtube.com>
- [12] <https://mem0.ai>
- [13] <https://www.digitalocean.com>